

ClassiRoute2

A Technical Report

of Training 104

BACHELOR OF TECHNOLOGY

Computer Science and Engineering

Submitted By:

[Student Name]

University Roll No.: [Roll No.]

Class Roll No.: [Class Roll No.]

Under the Guidance of

TnP Coordinators



Department of Computer Science and Engineering

**GURU NANAK DEV ENGINEERING COLLEGE,
LUDHIANA**

Abstract

ClassiRoute2 is an ML-powered, multi-provider LLM (Large Language Model) routing gateway that intelligently classifies and dispatches user prompts to the most cost-effective model tier. The system addresses a fundamental inefficiency in LLM application development: the one-size-fits-all approach to model selection. Production LLM applications typically commit to a single model provider, paying premium rates for simple queries while risking accuracy loss on complex tasks.

ClassiRoute2 solves this by introducing a three-tier model architecture—Weak, Mid, and Strong—where each tier can be configured with a different model provider (OpenAI-compatible endpoints, Anthropic Claude, or Google Gemini) and a different model variant (e.g., GPT-4o-mini for weak, Claude-3-Haiku for mid, Gemini-2.0-Pro for strong). An XGBoost-based classifier analyses incoming prompts across 22 linguistic and structural features to determine the optimal tier assignment. A regressor estimates per-prompt difficulty, and a confidence threshold mechanism ensures safe fallback to higher tiers when the classifier is uncertain.

The system exposes an OpenAI-compatible `/v1/chat/completions` endpoint, enabling drop-in replacement for existing OpenAI SDK integrations. A React-based web dashboard provides user authentication, virtual key management, an interactive playground, and comprehensive analytics visualising cost savings, latency distribution, tier utilisation, and success rates. A cascading fallback mechanism ensures robustness: if a provider for the assigned tier fails, the system automatically retries with progressively weaker (and cheaper) tiers without exposing errors to the end user.

The frontend is built with React 19, TypeScript, Vite, Tailwind CSS v4, and Radix UI primitives, deployed on Vercel. The backend is built with Python 3.13, FastAPI, SQLAlchemy (async), and PostgreSQL, deployed via Docker on Render. ML models are trained using scikit-learn and XGBoost on synthetically generated prompt corpora.

Acknowledgement

We would like to express our sincere gratitude to the Training and Placement (TnP) coordinators of Guru Nanak Dev Engineering College, Ludhiana, for their invaluable guidance and support throughout the development of this project. Their insights and encouragement have been instrumental in shaping the direction and quality of this work.

We extend our heartfelt thanks to the faculty members of the Department of Computer Science and Engineering for providing a strong academic foundation and the necessary infrastructure that enabled the successful completion of this project.

We are also grateful to our peers and colleagues whose constructive feedback during the development and testing phases helped refine the system's design and user experience.

List of Figures

1.1	ClassiRoute2 System Architecture Overview	12
3.1	Deployment Diagram Showing Component Distribution Across Services . .	22
3.2	Prompt Routing Pipeline — From Request Ingestion to Provider Dispatch	23
3.3	UML Class Diagram for the Provider Adapter Pattern	24
3.4	Entity-Relationship Diagram for the ClassiRoute2 Database	27
3.5	Authentication and Authorisation Flow Diagram	31
5.1	Dashboard Page Showing Analytics Summary and Tier Distribution	42
5.2	Chat Playground Showing Streaming Response with Routing Metadata . .	43
5.3	API Keys Management Page with Create Dialog	43
5.4	Detailed Analytics View with Request Logs and Daily Trends	44
5.5	Analytics Charts Showing Cost Savings and Tier Utilisation	44
5.6	Users Table in PostgreSQL	45
5.7	Virtual Keys Table Showing Encrypted API Key Storage	46
5.8	Request Logs Table Capturing Per-Request Routing and Usage Data . . .	46

List of Tables

3.1	Users Table Schema	27
3.2	Virtual Keys Table Schema	28
3.3	Request Logs Table Schema	29
3.4	API Route Map	30
3.5	Frontend Component Hierarchy and Routing	31
4.1	Technology Stack Summary	33
4.2	Backend Test Cases	39
4.3	Frontend Test Cases	40

Contents

Abstract	1
Acknowledgement	2
List of Figures	3
List of Tables	4
Table of Contents	7
1 Introduction	8
1.1 Introduction to Project	8
1.2 Project Category	8
1.3 Problem Formulation	9
1.4 Identification/Recognition of Need	9
1.5 Existing System	10
1.6 Objectives	11
1.7 Proposed System	11
1.8 Unique Features of the Proposed System	13
2 Requirement Analysis and System Specification	15
2.1 Feasibility Study	15
2.1.1 Technical Feasibility	15
2.1.2 Economical Feasibility	15
2.1.3 Operational Feasibility	16
2.2 Software Requirement Specification	16
2.2.1 Data Requirements	16
2.2.2 Functional Requirements	16
2.2.3 Performance Requirements	18
2.2.4 Dependability Requirements	18
2.2.5 Maintainability Requirements	18
2.2.6 Security Requirements	19

2.2.7	Look and Feel Requirements	19
2.3	SDLC Model to be Used	19
3	System Design	21
3.1	Design Approach	21
3.2	Detailed Design	22
3.2.1	System Architecture Overview	22
3.2.2	Prompt Routing Pipeline	23
3.2.3	Provider Adapter Architecture	24
3.2.4	Dispatch and Fallback Mechanism	25
3.2.5	Database Schema Design	26
3.2.6	API Route Design	29
3.2.7	Authentication and Authorisation Flow	30
3.2.8	Frontend Component Architecture	31
3.2.9	Rate Limiting Design	32
4	Implementation and Testing	33
4.1	Introduction to Languages, IDEs, Tools and Technologies	33
4.2	Algorithm/Pseudocode Used	34
4.2.1	Feature Extraction Algorithm	34
4.2.2	ML Routing Algorithm	35
4.2.3	Cascading Fallback Algorithm	36
4.2.4	Analytics Aggregation Algorithm	36
4.3	Testing Techniques	37
4.3.1	Testing Strategy	37
4.3.2	Tests Configuration	38
4.4	Test Cases Designed	39
5	Results and Discussions	41
5.1	User Interface Representation	41
5.1.1	Brief Description of Various Modules of the System	41
5.2	Snapshots of the System	42

5.3	Backend Representation	45
5.3.1	Database Tables	45
5.3.2	Cost Savings Analysis	47
6	Conclusion and Future Scope	48
6.1	Conclusion	48
6.2	Future Scope	49
	References/Bibliography	50

Chapter 1

Introduction

1.1 Introduction to Project

ClassiRoute2 is a self-hosted, ML-powered API gateway that sits between end-user applications and multiple Large Language Model (LLM) providers. Its primary function is to analyse incoming text prompts, classify them by complexity, and route them to the most appropriate model tier—thereby optimising the trade-off between response quality and operational cost.

The project was conceived in response to a growing problem in the LLM application ecosystem: as the number of available models expands across providers like OpenAI, Anthropic, and Google, developers face a combinatorial explosion of choices. Committing to a single model for all queries is suboptimal—simple tasks (e.g., summarisation, translation, simple Q&A) are over-served by frontier models at high cost, while complex tasks (e.g., code generation, multi-step reasoning, architectural design) require the most capable (and expensive) models available.

ClassiRoute2 introduces a three-tier abstraction layer—Weak, Mid, Strong—that lets developers configure exactly which model from which provider powers each tier. An ML classifier built on XGBoost [1] analyses prompt text across 22 linguistic and structural features to predict the optimal tier. A regressor estimates a continuous difficulty score, and a confidence-threshold safety mechanism ensures that ambiguous prompts are automatically escalated to higher tiers.

The system exposes a single OpenAI-compatible `/v1/chat/completions` endpoint, making it a drop-in replacement for any application already using the OpenAI SDK. This compatibility-by-design eliminates migration friction.

1.2 Project Category

ClassiRoute2 falls under the **Application Development** category. It is a full-stack web application with an embedded machine learning component that serves as a practical, deployable tool for developers and organisations using LLM APIs. The project spans the

following domains:

- **Full-Stack Web Development:** React 19 frontend (TypeScript, Vite, Tailwind CSS v4) and Python 3.13 backend (FastAPI, SQLAlchemy, PostgreSQL).
- **Machine Learning:** Supervised classification (XGBoost) for prompt tier assignment and regression for difficulty estimation, trained on synthetic prompt corpora.
- **API Gateway Design:** OpenAI-compatible REST endpoints, provider abstraction layer with adapter pattern, rate limiting, caching, and telemetry.
- **DevOps & Deployment:** Docker Compose for local development, Render for back-end hosting, Vercel for frontend hosting, automated Alembic database migrations.

1.3 Problem Formulation

Organisations integrating LLMs into their products face three interconnected challenges:

1. **Cost Inefficiency:** Frontier models (GPT-4o, Claude-3.5 Sonnet, Gemini-2.0-Pro) charge \$2–\$15 per million input tokens. A chat application handling millions of queries—where 70%+ are simple retrievals or short-form Q&A—pays premium rates for trivial work.
2. **Provider Lock-In:** Building directly against a single provider’s SDK couples the application to that provider’s API semantics, error handling, and rate limits. Migrating to a different provider or model requires rewiring the entire integration layer.
3. **No Intelligent Routing:** Existing solutions either use a fixed model for all traffic (wasteful) or require the developer to manually implement routing logic (error-prone and inflexible). No widely adopted open-source solution uses ML to automate tier assignment dynamically.

ClassiRoute2 addresses all three challenges: it reduces costs by routing simple queries to cheap models, eliminates provider lock-in via a uniform adapter interface, and automates routing decisions using ML classification.

1.4 Identification/Recognition of Need

The need for intelligent LLM routing emerged from analysing usage patterns of production LLM applications. A typical AI-powered customer support chatbot, for example, handles:

- **Simple queries (60–70% of traffic):** Password reset instructions, business hours, order status checks. These require basic language understanding and can be handled by small, cheap models.
- **Moderate queries (20–25%):** Product comparisons, troubleshooting steps, policy clarifications. These benefit from mid-tier models with better reasoning.
- **Complex queries (5–10%):** Contract analysis, code review, multi-step workflow planning. These demand frontier-model capability.

Without intelligent routing, all queries are handled by the most expensive model, wasting 60–90% of the API budget. The industry need is clear: a drop-in gateway that learns to discriminate these tiers automatically.

1.5 Existing System

Several approaches exist for managing LLM API usage, each with limitations:

Direct SDK Integration: Applications call provider SDKs (OpenAI, Anthropic, etc.) directly. This is the simplest approach but offers no routing, no provider abstraction, and no cost optimisation. Developers must manually choose which model to call for each request.

API Key Rotation / Load Balancing: Tools like Helicone and MLGateway distribute requests across multiple API keys for rate-limit management but do not perform content-based routing.

Manual Tier Selection: Some applications implement a simple if-else heuristic (e.g., “if prompt length > 1000 tokens, use strong model”). These heuristics are brittle, require constant adjustment, and fail to capture semantic complexity.

Managed Gateways (Azure API Management, Kong): Enterprise API gateways provide rate limiting, caching, and authentication but lack LLM-specific features like prompt analysis, provider abstraction, and cost analytics.

No existing open-source solution combines ML-based prompt routing, multi-provider support with a unified adapter interface, cascading fallback, and comprehensive cost analytics in a single deployable package.

1.6 Objectives

The project was guided by the following specific objectives:

1. Design and implement a three-tier LLM routing architecture (Weak, Mid, Strong) with configurable providers and models per tier.
2. Develop an ML-based prompt classifier using XGBoost that achieves >85% accuracy in assigning prompts to appropriate tiers based on linguistic and structural features.
3. Implement a provider adapter pattern supporting OpenAI-compatible endpoints (NVIDIA NIM, Together AI, Groq, etc.), Anthropic Claude, and Google Gemini with normalised request/response formats.
4. Build a cascading fallback mechanism that automatically retries failed provider calls across tiers to ensure zero-downtime request processing.
5. Create a full-stack web dashboard with user authentication, virtual API key management, an interactive chat playground, and real-time analytics visualising cost savings, latency, and tier utilisation.
6. Expose an OpenAI-compatible `/v1/chat/completions` endpoint to enable drop-in replacement for existing OpenAI SDK integrations.
7. Implement rate limiting, request telemetry (PostHog), and error tracking (Sentry) for production observability.
8. Containerise the application using Docker and provide deployment configurations for Render and Vercel.

1.7 Proposed System

ClassiRoute2 proposes a unified LLM gateway architecture where:

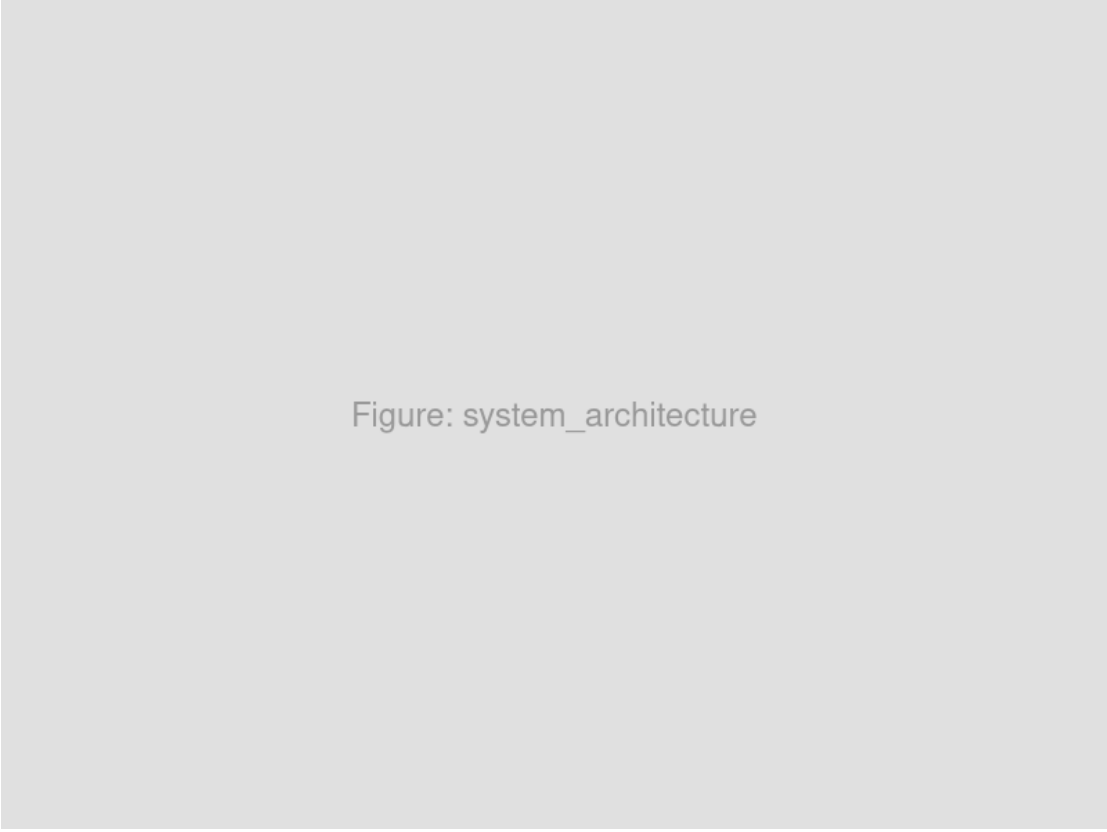


Figure: system_architecture

Figure 1.1: ClassiRoute2 System Architecture Overview

The system comprises five principal components:

1. **Frontend Dashboard (React SPA):** A single-page application providing user authentication, virtual key management, an interactive chat playground with streaming responses, and analytics dashboards with Recharts visualisations.
2. **API Gateway (FastAPI Backend):** The core request-processing pipeline—authentication → prompt feature extraction → ML classification → provider dispatch → response normalisation → telemetry logging.
3. **Provider Adapters:** A polymorphic adapter layer that abstracts provider-specific SDK differences behind a common **BaseProvider** interface. Each adapter normalises request formats, response structures, error codes, and usage metadata.
4. **ML Routing Engine:** An XGBoost classifier and regressor trained on a feature space of 22 dimensions (complexity score, code patterns, reasoning indicators, readability metrics, etc.) that predicts optimal tier assignment and assigns a confidence score.
5. **PostgreSQL Data Layer:** Persistent storage for user accounts, virtual keys (with Fernet-encrypted API secrets), request logs, and analytics aggregation.

1.8 Unique Features of the Proposed System

ClassiRoute2 offers the following distinctive capabilities that differentiate it from existing solutions:

- **ML-Powered Tier Assignment:** Unlike static rule-based routing, ClassiRoute2 uses a trained XGBoost classifier that learns from prompt features to predict optimal tier assignment. The classifier is complemented by a regressor that estimates per-prompt difficulty, and a confidence-threshold mechanism that automatically upgrades uncertain predictions.
- **Multi-Provider Adapter Architecture:** The system supports OpenAI-compatible endpoints, Anthropic Claude, and Google Gemini through a polymorphic adapter pattern. Adding a new provider requires implementing only the `BaseProvider` interface—no core routing logic changes. The registry pattern (`register_provider/get_provider`) enables zero-config provider registration via import side effects.
- **Cascading Fallback Resilience:** If a provider call to the assigned tier fails (due to rate limits, API errors, or network issues), the system automatically retries on progressively weaker tiers. The fallback is transparent to the client, which receives a routing metadata header (`x-llmrouter`) indicating the rerouting.
- **OpenAI-Compatible Endpoint:** The `/v1/chat/completions` endpoint mirrors the OpenAI API shape exactly, including streaming chunks, usage metadata, and error formats. Any application using the OpenAI Python/Node.js SDK can switch to ClassiRoute2 by changing only the base URL and API key.
- **Virtual Key System with Shamir-Style Secret Management:** Each user creates virtual keys that bundle three API credentials (one per tier). The actual API keys are encrypted at rest using Fernet symmetric encryption. The system returns a human-readable key (`clr-...`) that is stored hashed (SHA-256) for lookup, never in plaintext.
- **Dynamic Provider Verification:** Before creating a virtual key, the system proactively verifies that each configured model exists on the provider's endpoint by querying the `/models` API (OpenAI-compatible) or equivalent, providing immediate validation feedback.

- **Comprehensive Cost Analytics:** The analytics dashboard computes not just raw usage but also cost-saved-vs-always-strong, enabling users to quantify the ROI of intelligent routing. Analytics are broken down by tier, by day, and by individual key.

Chapter 2

Requirement Analysis and System Specification

2.1 Feasibility Study

2.1.1 Technical Feasibility

The project leverages mature, well-supported technologies. On the backend, Python 3.13 with FastAPI provides a high-performance async web framework. SQLAlchemy’s async ORM with PostgreSQL offers robust data persistence with connection pooling. On the frontend, React 19 with TypeScript provides type-safe component architecture, while Vite ensures rapid development iteration via HMR (Hot Module Replacement).

The ML component uses scikit-learn and XGBoost, both battle-tested libraries with extensive documentation. The model files (`router_classifier.pkl`, `router_regressor.pkl`) are serialised using Python’s pickle format, loaded at application startup. In the absence of model files, the system gracefully degrades to heuristic routing based on a complexity score threshold.

All dependencies are available via PyPI and npm, with locked versions in `uv.lock` and `package.json` respectively. Docker images are based on `python:3.13-slim` with multi-stage builds for minimal footprint.

2.1.2 Economical Feasibility

The project is entirely open-source and uses free-tier or low-cost infrastructure:

- **Backend host:** Render free tier (512 MB RAM, shared CPU) is sufficient for low-traffic deployments.
- **Frontend host:** Vercel free tier provides generous bandwidth and global CDN.
- **Database:** PostgreSQL can be self-hosted via Docker or provisioned via Render’s managed PostgreSQL (\$7/month) or Supabase free tier.
- **ML training:** No GPU required—XGBoost training on CPU takes seconds for the synthetic dataset.

The total monthly cost for a production deployment is approximately \$0–\$15, making the system economically viable for startups and small teams.

2.1.3 Operational Feasibility

The system is designed for minimal operational overhead. Alembic migrations run automatically on application startup (via the FastAPI lifespan hook). Container startup includes a database-wait script that polls the PostgreSQL port before launching. Health checks (`/health`) verify database connectivity.

Logging is structured and tiered: Python’s `logging` module for system logs, Sentry for error tracking (with noise filtering for 404/422 responses), and PostHog for product telemetry. All features are gated by environment variables and degrade gracefully when optional services (PostHog, Sentry) are not configured.

2.2 Software Requirement Specification

2.2.1 Data Requirements

The system persists four primary entity types:

- **User:** Email (unique, indexed), username, bcrypt-hashed password, admin flag, creation timestamp.
- **VirtualKey:** Owner (foreign key to User), name, SHA-256 hash of the raw key, three tiers of model configuration (model ID, Fernet-encrypted API key, base URL, provider type), active flag, timestamps for creation and last use.
- **RequestLog:** Links to VirtualKey and User, prompt preview (truncated to 200 chars), prompt length, assigned tier, confidence score, model used, token counts, latency (ms), cost estimate (USD), status, error message, timestamp.
- **Analytics (computed):** Aggregated from RequestLog—total requests, requests by tier, total cost, cost saved vs. always-strong, average latency, success rate, daily breakdowns.

2.2.2 Functional Requirements

1. **FR1 - User Registration:** The system shall allow new users to register with email, username, and password (minimum 8 characters). Passwords shall be hashed using bcrypt. Duplicate email registration shall be rejected.

2. **FR2 - User Authentication:** The system shall issue JWT tokens (default 30-day expiry, HS256 signing) upon successful login. Token validation shall occur on all protected API routes. The `/auth/me` endpoint shall return the authenticated user's profile.
3. **FR3 - Virtual Key Management:** Authenticated users shall be able to create, list, and revoke virtual keys. Each key bundles three tier configurations. The raw key (`clr-...`) shall be displayed exactly once upon creation. API keys at rest shall be encrypted using Fernet symmetric encryption.
4. **FR4 - Provider Verification:** Before creating a key, the system shall verify that the configured model exists on the specified provider's endpoint. For OpenAI-compatible providers, this is done via `GET /models`. Anthropic and Gemini have dedicated verification endpoints.
5. **FR5 - Chat Completion:** The `/v1/chat/completions` endpoint shall accept OpenAI-format requests and return OpenAI-format responses (both streaming and non-streaming). The endpoint shall authenticate via `Bearer clr-...` tokens.
6. **FR6 - ML-Based Routing:** Each incoming prompt shall be analysed by the feature extractor. If the ML classifier is loaded, it shall predict the optimal tier. If confidence is below 0.60, the tier shall be upgraded by one level. If the classifier is unavailable, heuristic routing based on complexity score shall be used.
7. **FR7 - Cascading Fallback:** If a provider call fails on the assigned tier, the system shall retry the request on progressively weaker tiers (e.g., tier $2 \rightarrow 1 \rightarrow 0$) until success or exhaustion.
8. **FR8 - Rate Limiting:** The system shall enforce sliding-window rate limits: 60 requests/minute for chat, 20/min for auth, 200/min for general API, and 100/min for admin endpoints.
9. **FR9 - Analytics:** Authenticated users shall be able to view summary statistics (total requests, cost, latency, success rate), request logs (paginated, filterable by key), and daily usage trends. Cost-saved-vs-always-strong shall be computed and displayed.
10. **FR10 - User Profile Management:** Authenticated users shall be able to update their username, email, and password via the `/users/me` endpoint.
11. **FR11 - Admin User Management:** Admin users shall be able to list all registered

users with pagination.

2.2.3 Performance Requirements

- API response time for routing classification shall not exceed 50ms (including feature extraction).
- Database query latency for analytics aggregation over 30-day windows shall complete within 500ms.
- The system shall handle a minimum of 50 concurrent streaming connections.
- Page load for the analytics dashboard shall complete within 2 seconds on a standard broadband connection.

2.2.4 Dependability Requirements

- Database connection shall use connection pooling (10 pool size, 10 overflow) with `pool_pre_ping=True` to detect stale connections.
- Connection recycling shall occur every 30 minutes to prevent cloud-PG idle connection termination.
- The API shall include a `/health` endpoint that reports database connectivity status.
- Cascading fallback shall ensure that a single provider outage does not cascade to application-level failure.

2.2.5 Maintainability Requirements

- Database schema changes shall be managed via Alembic migrations with versioned, reversible migrations.
- Provider adapters shall implement the `BaseProvider` abstract interface, enabling new providers to be added without modifying core routing or dispatch logic.
- Feature extraction shall be a pure function operating on text input, independently testable and replaceable.
- Environment configuration shall be managed via `.env` files with documented defaults in `.env.example`.

2.2.6 Security Requirements

- Passwords shall be hashed using bcrypt via the `passlib` library.
- JWT tokens shall use HS256 with a minimum 64-character secret key.
- Provider API keys shall be encrypted at rest using Fernet symmetric encryption (AES-128-CBC with HMAC-SHA256 authentication).
- Virtual keys shall be stored as SHA-256 hashes; the raw value is never persisted.
- CORS shall be restricted to configured origins (default: localhost:5173, localhost:3000).
- Rate limiting on auth endpoints shall mitigate brute-force attacks.

2.2.7 Look and Feel Requirements

- The UI shall use a dark colour scheme with consistent brand tokens (orange#d97757, blue#6a9bcc, green#788c5d).
- The layout shall feature a collapsible sidebar navigation with icons (Lucide iconset) and responsive breakpoints.
- All data tables shall support loading, empty, and error states.
- Streaming chat output shall render markdown with syntax highlighting (highlight.js via react-markdown and rehype-highlight).
- Analytics charts shall use Recharts with custom-branded tooltips.

2.3 SDLC Model to be Used

The project was developed following an **Iterative and Incremental Development** model, which aligns with the practical constraints of a solo/team academic project. The development proceeded in six iterations:

1. **Iteration 1 - Core API:** FastAPI server with a single `/chat` endpoint, basic provider adapter (NVIDIA NIM), and health check.
2. **Iteration 2 - Authentication:** User registration, JWT token issuance, password hashing, protected routes.
3. **Iteration 3 - Virtual Keys & Multi-Provider:** Virtual key CRUD, Fernet encryption, provider verification, OpenAI and Anthropic adapters.
4. **Iteration 4 - ML Routing:** Feature extraction pipeline, XGBoost classifier/regressor

training, heuristic fallback, confidence upgrade mechanism.

5. **Iteration 5 - Frontend Dashboard:** React SPA with login/register, key management, chat playground, and analytics visualisation.
6. **Iteration 6 - Production Hardening:** Rate limiting, Sentry error tracking, PostHog telemetry, Docker Compose, Alembic migration automation, test suite.

Each iteration concluded with working, deployable software. The incremental approach allowed early validation of the core routing concept before investing in the UI layer, and enabled parallel work on the ML components and frontend.

Chapter 3

System Design

3.1 Design Approach

The system employs an **Object-Oriented Design (OOD)** approach combined with a **Layered Architecture** pattern. The design is driven by three principles:

1. **Separation of Concerns:** Each layer (API routes, business logic, data access, provider integration) has distinct responsibilities. Inter-layer communication occurs through well-defined interfaces (Pydantic models for data transfer, abstract base classes for provider integration).
2. **Dependency Inversion:** High-level routing logic depends on the `BaseProvider` abstraction, not concrete provider implementations. Provider classes are registered via a registry pattern and instantiated dynamically at dispatch time.
3. **Open/Closed Principle:** New providers can be added by implementing the `BaseProvider` interface and importing the module (which triggers self-registration). No routing, dispatch, or API-layer code needs modification.

3.2 Detailed Design

3.2.1 System Architecture Overview

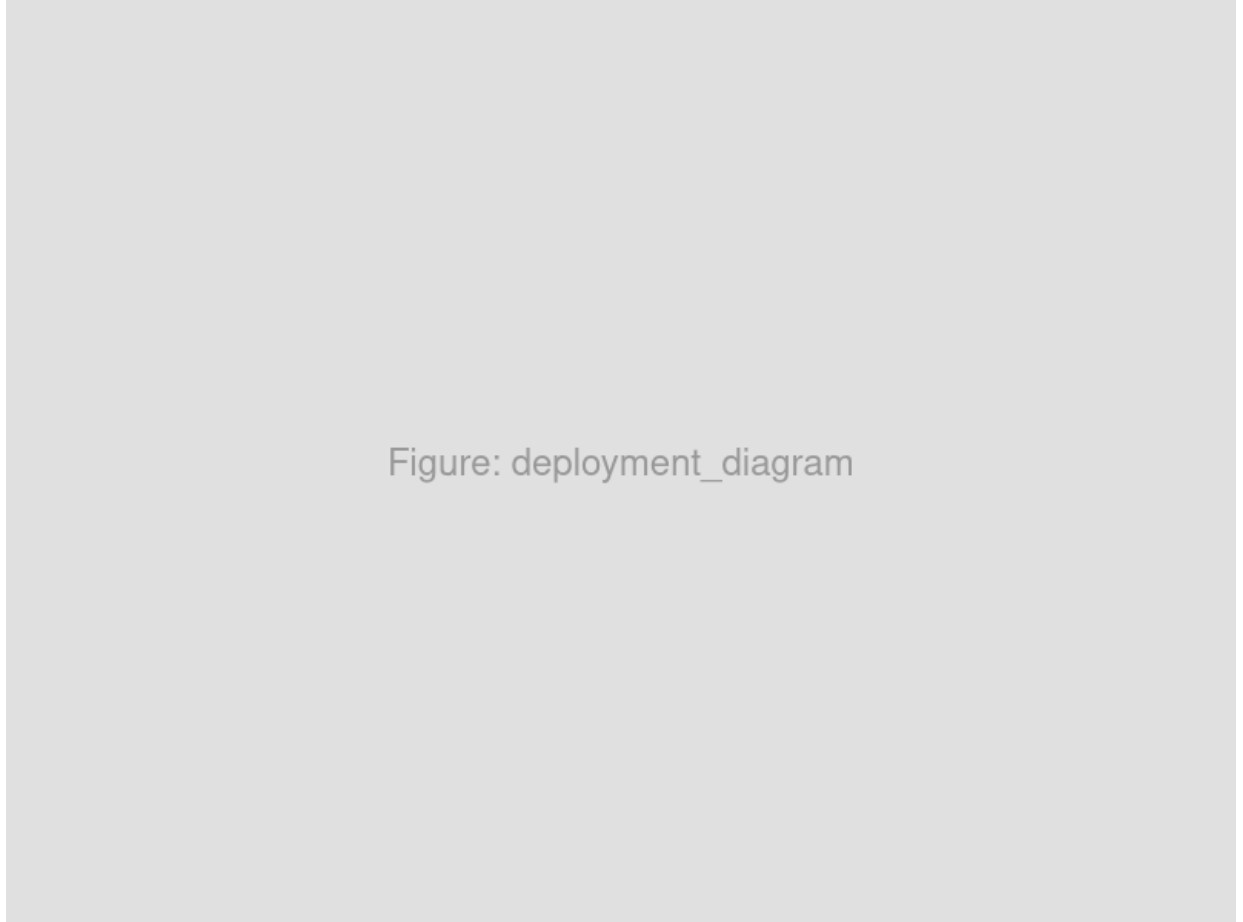


Figure 3.1: Deployment Diagram Showing Component Distribution Across Services

The system follows a client-server architecture with three deployment nodes:

1. **Vercel (Frontend):** Hosts the React SPA. API requests to `/v1/*`, `/auth/*`, `/keys/*`, and `/analytics/*` are proxied to the backend via Vite’s dev server (in development) or Vercel’s rewrites (in production).
2. **Render / Docker (Backend):** Hosts the FastAPI application. In production, Render runs `uvicorn main:app` with the `startCommand` defined in `render.yaml`. In development, Docker Compose orchestrates the backend container alongside a PostgreSQL instance.
3. **PostgreSQL (Database):** Stores all persistent data. Run as a sidecar container in development (`postgres:16-alpine`), managed as a Render-managed PostgreSQL in production.

3.2.2 Prompt Routing Pipeline

The core request flow through the system can be visualised as follows:



Figure 3.2: Prompt Routing Pipeline — From Request Ingestion to Provider Dispatch

The routing pipeline (implemented in `core/router.py`) executes the following decision logic:

1. **Feature Extraction (`core/feature_extractor.py`):** The raw prompt text is analysed to produce a 22-dimensional feature vector. Features include:
 - **Lexical:** character count, word count, sentence count, average word length, unique word ratio, average sentence length, Flesch-Kincaid Grade Level.
 - **Semantic flags (binary):** `is_coding`, `is_debugging`, `is_reasoning`, `is_creative`, `is_multistep`, `is_math`, `is_summarize`, `is_simple_qa`.
 - **Structural:** has code blocks (`"'`), has numbers, question count, comma count, bullet points, constraint language (`must/require/limit/only/exactly`), capitalisation ratio.
 - **Composite:** `complexity_score` — a weighted sum of the above features nor-

malised to a 0–10 scale.

2. **ML Classification:** If the XGBoost classifier is loaded:
 - `CLASSIFIER.predict_proba(features)` returns a 3-element probability vector.
 - The tier with the highest probability is selected: `tier = argmax(probs)`.
 - `confidence = probs[tier]` is the model’s certainty.
 - If `confidence < 0.60` and `tier < 2`, the tier is incremented by 1 (conservative upgrade).
3. **Heuristic Fallback:** If the classifier is not loaded (model files missing or corrupt):
 - `complexity_score < 2.0` → tier 0 (weak)
 - `2.0 ≤ complexity_score < 4.0` → tier 1 (mid)
 - `complexity_score ≥ 4.0` → tier 2 (strong)
4. **Difficulty Regression:** The regressor (if loaded) predicts a continuous difficulty score. Otherwise, difficulty is estimated as `min(complexity_score / 10.0, 1.0)`.
5. **Routing Response:** Returned as a dictionary with `tier`, `tier_name`, `confidence`, `difficulty_score`, `upgraded`, and `rerouted` fields.

3.2.3 Provider Adapter Architecture

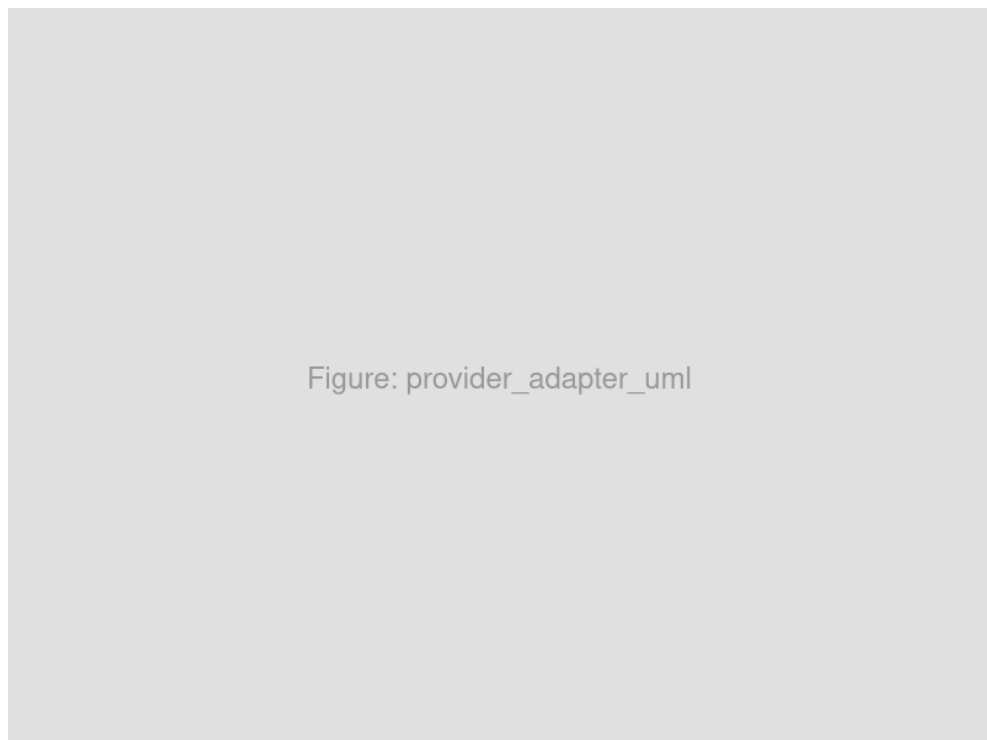


Figure: provider_adapter_uml

Figure 3.3: UML Class Diagram for the Provider Adapter Pattern

The provider system is built on four design elements:

1. **Abstract Base (core/providers/base.py):** Declares two abstract methods: `chat(message, model, api_key, **kwargs) -> dict` for synchronous responses and `chat_stream(...)` -> `AsyncGenerator` for streaming. Both return normalised formats.
2. **Concrete Adapters:** Each provider (OpenAI-compatible, Anthropic, Gemini) implements `BaseProvider`:
 - **OpenAIProvider:** Uses the `openai.AsyncOpenAI` SDK. Handles any OpenAI-compatible endpoint (NVIDIA NIM, Together, Groq, etc.) via configurable `base_url`.
 - **AnthropicProvider:** Uses the `anthropic.AsyncAnthropic` SDK. Extracts the system message from the messages array (required as a top-level field in Anthropic's API). Converts message format to Anthropic's schema.
 - **GeminiProvider:** Uses the `google.genai` SDK. Converts messages to Gemini's content format. Handles system instructions via `GenerateContentConfig`.
3. **Registry (core/providers/registry.py):** A module-level dictionary (`PROVIDER_MAP`) mapping provider name strings to provider classes. Registration occurs via the `register_provider()` function, called as a module import side effect in each adapter file.
4. **Dispatch (core/dispatcher.py):** The `dispatch_stream()` and `dispatch_sync()` functions accept messages, a `VirtualKey` object, and a tier integer. They dynamically select the appropriate attribute (e.g., `virtual_key.weak_model`, `virtual_key.weak_api_key`), decrypt the API key using Fernet, resolve the provider via `get_provider()`, and invoke the normalised interface.

3.2.4 Dispatch and Fallback Mechanism

The cascading fallback logic (implemented in `api/v1/chat.py`) operates as follows:

```

1 original_tier = routing["tier"]
2 attempts = list(range(original_tier, -1, -1))
3
4 for attempt in attempts:
5     try:
6         stream_obj, model_used = await dispatch_stream(

```

```
7         messages, virtual_key, attempt
8     )
9     if attempt != original_tier:
10         # Mark rerouting metadata
11         routing["rerouted"] = True
12         routing["original_tier"] = original_tier
13         routing["tier"] = attempt
14         yield stream chunks with routing metadata
15         break
16     except Exception:
17         if attempt == 0:
18             yield error
19         else:
20             continue # fallback to next tier
```

Listing 3.1: Cascading Fallback Algorithm

This ensures that if the assigned tier’s provider is unreachable, the system gracefully degrades to a cheaper tier rather than returning an error. The client receives routing metadata (via the `x-llmrouter` header/field in the response body) indicating the original assigned tier and the final serving tier.

3.2.5 Database Schema Design

The database schema comprises three tables connected through foreign key relationships:

Figure: er_diagram

Figure 3.4: Entity-Relationship Diagram for the ClassiRoute2 Database*Users Table***Table 3.1:** Users Table Schema

Column	Type	Constraints
id	UUID	PK, default uuid4
email	String	UNIQUE, NOT NULL, INDEX
username	String	NOT NULL
password_hash	String	NOT NULL
is_admin	Boolean	DEFAULT false
created_at	DateTime	DEFAULT now()

*Virtual Keys Table***Table 3.2:** Virtual Keys Table Schema

Column	Type	Constraints
id	UUID	PK
user_id	UUID	FK → users.id, ON DELETE CASCADE
name	String	NOT NULL
key_hash	String	UNIQUE, NOT NULL, INDEX
weak_model	String	NOT NULL
weak_api_key	String	NOT NULL (Fernet-encrypted)
weak_base_url	String	NULLABLE
weak_provider_type	String(20)	DEFAULT 'openai'
mid_model	String	NOT NULL
mid_api_key	String	NOT NULL (Fernet-encrypted)
mid_base_url	String	NULLABLE
mid_provider_type	String(20)	DEFAULT 'openai'
strong_model	String	NOT NULL
strong_api_key	String	NOT NULL (Fernet-encrypted)
strong_base_url	String	NULLABLE
strong_provider_type	String(20)	DEFAULT 'openai'
is_active	Boolean	DEFAULT true
created_at	DateTime	DEFAULT now()
last_used_at	DateTime	NULLABLE

*Request Logs Table***Table 3.3:** Request Logs Table Schema

Column	Type	Constraints
id	UUID	PK
virtual_key_id	UUID	FK → virtual_keys.id
user_id	UUID	FK → users.id, ON DELETE CASCADE
prompt_preview	String(200)	NULLABLE
prompt_length	Integer	NOT NULL
tier_assigned	Integer	NOT NULL
confidence	Float	NOT NULL
model_used	String	NOT NULL
input_tokens	Integer	NULLABLE
output_tokens	Integer	NULLABLE
latency_ms	Integer	NOT NULL
cost_estimate_usd	Float	NULLABLE
status	String	NOT NULL
error_message	String	NULLABLE
created_at	DateTime	DEFAULT now()

3.2.6 API Route Design

The system exposes the following route groups:

Table 3.4: API Route Map

Prefix	Methods	Description
/auth	POST register, POST login, GET me	Authentication
/v1/chat/completions	POST	Chat completions (streaming & sync)
/keys	POST create, POST revoke, GET list	Virtual key CRUD
/v1/keys/anthropic	POST verify, POST models	Anthropic verification
/v1/keys/gemini	POST verify, POST models	Gemini verification
/analytics	GET summary, GET requests, GET daily	Analytics aggregation
/users	GET /, GET me, GET {_id}, PUT me	User management
/v1/internal	POST evaluate	Routing evaluation
/health	GET	Health check

3.2.7 Authentication and Authorisation Flow

The system implements a dual authentication strategy:

1. **JWT Token Auth (Web UI):** Used for all frontend-facing routes (`/keys/*`, `/analytics/*`, `/auth/me`, `/users/*`). The `get_current_user` dependency extracts the Bearer token from the `Authorization` header, decodes the JWT (validating signature and expiry), looks up the user by the `sub` claim (user UUID), and injects the `User` object into the route handler.
2. **Virtual Key Auth (API):** Used exclusively for the `/v1/chat/completions` endpoint. The `get_virtual_key` dependency expects a Bearer token starting with `clr-`. It SHA-256 hashes the raw key, looks up the hash in the `virtual_keys` table, verifies

the key is active, and injects the `VirtualKey` object into the handler.

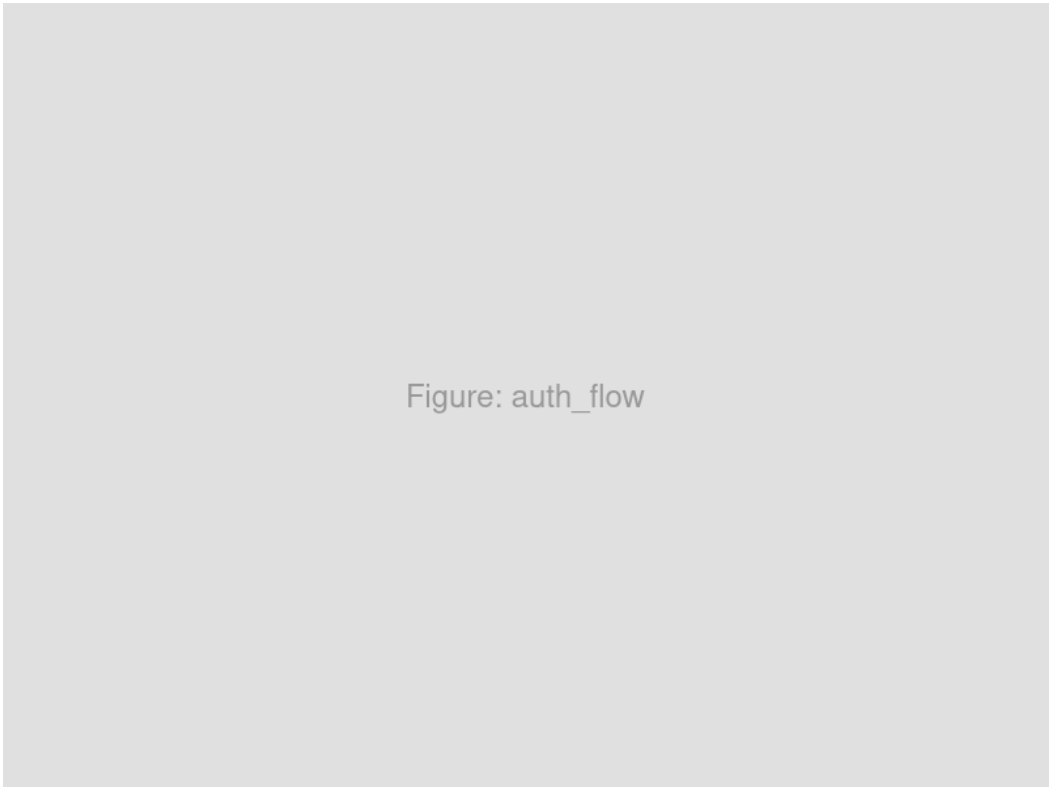


Figure 3.5: Authentication and Authorisation Flow Diagram

3.2.8 Frontend Component Architecture

Table 3.5: Frontend Component Hierarchy and Routing

Component	Route	Description
App	-	Root router with lazy-loaded pages
Layout	(wrapper)	Sidebar + Outlet, auth redirect
Login	/login	Email/password form
Register	/register	Registration form
Dashboard	/	Analytics summary cards + charts
Keys	/keys	Virtual key CRUD with provider tabs
Analytics	/analytics	Detailed analytics with filters
Chat	/chat	Interactive chat playground
Profile	/profile	User profile management

3.2.9 Rate Limiting Design

The `SlidingWindowRateLimiter` (in `core/rate_limiter.py`) implements an in-memory sliding window algorithm:

- Each rate limit key (e.g., `chat:{user_id}`) maintains a sorted list of Unix timestamps representing request times.
- On each request, timestamps older than `window_seconds` are evicted ("cleanup").
- If the count of remaining timestamps exceeds `max_requests`, the request is rejected with HTTP 429.
- The `Retry-After` header is computed from the oldest timestamp in the window.

The rate limiter uses `asyncio.Lock` for thread-safe concurrent access in the async context. Rate limits are configured per endpoint group:

- Chat: 60 requests per 60 seconds per user
- Auth: 20 requests per 60 seconds per IP
- API: 200 requests per 60 seconds per user
- Admin: 100 requests per 60 seconds per user

Chapter 4

Implementation and Testing

4.1 Introduction to Languages, IDEs, Tools and Technologies

Table 4.1: Technology Stack Summary

Category	Technology	Purpose
Backend Runtime	Python 3.13	Async runtime, async/await concurrency
Web Framework	FastAPI 0.136+	High-performance async REST framework
ORM	SQLAlchemy 2.0 (async)	Async ORM with connection pooling
Database	PostgreSQL 16	Relational data store
Frontend Framework	React 19	Component-based UI library
Build Tool	Vite 8	Dev server with HMR, production bundling
Styling	Tailwind CSS v4	Utility-first CSS framework
UI Components	Radix UI + shadcn/ui	Accessible, unstyled primitives
ML Framework	XGBoost 3.2	Gradient-boosted decision trees
Feature Extraction	textstat 0.7	Readability metrics
Authentication	python-jose	JWT encoding/decoding
Encryption	cryptography (Fernet)	Symmetric API key encryption
Password Hashing	passlib (bcrypt)	Password hashing
Containerisation	Docker + Docker Compose	Local dev environment
CI/CD	Render + Vercel	Production hosting
Monitoring	Sentry + PostHog	Error tracking, product telemetry
Testing	pytest + vitest	Backend and frontend testing

IDE: VS Code with Python and TypeScript extensions, Pylance for type checking, and ESLint for linting.

Version Control: Git with GitHub for source code management.

API Testing: curl, httpx (for integration tests), and the built-in Swagger UI at `/docs`.

Database Migration: Alembic for schema versioning, with auto-generated migration stubs from SQLAlchemy model changes.

4.2 Algorithm/Pseudocode Used

4.2.1 Feature Extraction Algorithm

The feature extractor (`core/feature_extractor.py`) computes a 22-dimensional feature vector from raw prompt text. The algorithm proceeds in the following stages:

1. **Lexical Analysis:** Character count, word count (via `textstat.lexicon_count`), sentence count (via `textstat.sentence_count`), average word length, unique word ratio using `re.findall(r'\b\w+\b', text_lower)`.
2. **Readability Scoring:** Flesch-Kincaid Grade Level is computed via `textstat.flesch_kincaid_grade_level`.
3. **Pattern Detection (binary classification):** Eight regex-based semantic detectors operate on the lowercased text:
 - `is_coding`: matches `\b(code|implement|function|class|algorithm|...)\b`
 - `is_debugging`: matches `\b(debug|error|traceback|exception|fix|bug|...)\b`
 - `is_reasoning`: matches `\b(explain|why|how does|analyze|compare|...)\b`
 - `is_creative`: matches `\b(poem|story|creative|imagine|fiction|...)\b`
 - `is_multistep`: matches `\b(design|architecture|plan|scalable|system|...)\b`
 - `is_math`: matches `\b(solve|calculate|equation|integral|...)\b`
 - `is_summarize`: matches `\b(summarize|summary|tldr|brief|overview|...)\b`
 - `is_simple_qa`: matches `\b(what is|who is|when did|where is|capital of|...)\b`
4. **Structural Indicators:** Code block presence (`"`"`), number presence, question count, comma count, bullet presence (lines starting with `-` or `*`), constraint language (`must/require/`), and capitalisation ratio.
5. **Complexity Score:** A weighted composite score computed from the above features, normalised to a 0–10 scale. Higher values indicate prompts requiring more capable

models.

4.2.2 ML Routing Algorithm

```
1 Input: prompt_text (string)
2 Output: routing_decision (dict)
3
4 1. features = extract_features(prompt_text)
5 2. feature_vector = get_feature_vector(prompt_text)
6
7 3. if CLASSIFIER is not loaded:
8 4.     if features.complexity_score < 2.0:
9 5.         tier = 0
10 6.     else if features.complexity_score < 4.0:
11 7.         tier = 1
12 8.     else:
13 9.         tier = 2
14 10. return heuristic_result
15
16 11. # ML Classification
17 12. probs = CLASSIFIER.predict_proba([feature_vector])
18 13. tier = argmax(probs)
19 14. confidence = probs[tier]
20
21 15. # Difficulty Regression
22 16. if REGRESSOR is loaded:
23 17.     difficulty = REGRESSOR.predict([feature_vector])
24 18. else:
25 19.     difficulty = min(complexity_score / 10.0, 1.0)
26
27 20. # Confidence Upgrade
28 21. if confidence < 0.60 AND tier < 2:
29 22.     tier += 1
30 23.     upgraded = True
31
32 24. return {
33 25.     "tier": tier,
34 26.     "tier_name": TIER_NAMES[tier],
```

```

35 27.     "confidence": confidence,
36 28.     "difficulty_score": difficulty,
37 29.     "upgraded": upgraded,
38 30.     "rerouted": False
39 31. }

```

Listing 4.1: XGBoost Routing Algorithm

4.2.3 Cascading Fallback Algorithm

```

1 Input: messages, virtual_key, assigned_tier
2 Output: stream of response chunks or error
3
4 1. attempts = [assigned_tier, assigned_tier-1, ..., 0]
5 2. for each tier t in attempts:
6 3.     model = virtual_key.get_model(t)
7 4.     api_key = decrypt(virtual_key.get_api_key(t))
8 5.     provider = get_provider(virtual_key.get_provider_type(t))
9 6.     try:
10 7.         stream = provider.chat_stream(messages, model, api_key)
11 8.         for chunk in stream:
12 9.             if t != assigned_tier:
13 10.                 attach routing metadata (x-llmrouter)
14 11.                 yield chunk
15 12.                 break # success
16 13.     catch Exception:
17 14.         if t == 0:
18 15.             yield error
19 16.         else:
20 17.             continue # try next tier

```

Listing 4.2: Cascading Fallback Algorithm

4.2.4 Analytics Aggregation Algorithm

The analytics summarisation (in `db/crud.py`) computes per-user statistics over configurable time windows:

```
1 Input: user_id, key_id (optional), days (default 30)
2 Output: summary statistics dict
3
4 1. cutoff = now() - days
5 2. logs = SELECT * FROM request_logs
6     WHERE user_id = user_id
7     AND created_at >= cutoff
8     [AND virtual_key_id = key_id]
9
10 3. For each log in logs:
11     by_tier[log.tier_assigned] += 1
12     total_cost += log.cost_estimate_usd
13     total_latency += log.latency_ms
14     if log.status == "success": success_count += 1
15     cost_if_all_strong += calculate_strong_cost(log)
16
17 4. return {
18     "total_requests": len(logs),
19     "by_tier": by_tier,
20     "total_cost_usd": total_cost,
21     "cost_saved_vs_always_strong": cost_if_all_strong - total_cost,
22     "avg_latency_ms": total_latency / len(logs),
23     "success_rate": success_count / len(logs)
24 }
```

Listing 4.3: Analytics Aggregation Pseudocode

The `cost_saved_vs_always_strong` metric compares the actual cost (using tier-specific rates) against a hypothetical scenario where every request was served by the strongest tier at \$3.00 per million tokens.

4.3 Testing Techniques

4.3.1 Testing Strategy

The project employs a multi-layered testing strategy:

1. **Unit Tests (Backend):** pytest-based tests for individual modules—feature ex-

traction, password hashing, JWT token creation/validation, rate limiter logic, and database CRUD operations. These tests use mocked database sessions and do not require a running PostgreSQL instance.

2. **Integration Tests (Backend):** `pytest-asyncio` tests that exercise the full API stack against a running database and API server. The `conftest.py` creates temporary users via the `/auth/register` endpoint and uses `httpx.AsyncClient` for HTTP-level testing. Integration tests are gated behind the `RUN_INTEGRATION=1` environment variable.
3. **Unit Tests (Frontend):** Vitest tests for the API client module (`client.test.ts`)—testing JWT token attachment, error handling, and response parsing. `LocalStorage` is stubbed using `vi.stubGlobal`.
4. **Component Tests (Frontend):** Vitest with `jsdom` environment for UI component rendering tests. Test files co-located with components (`ChatInput.test.tsx`, `ConfirmationDialog.test.tsx`, `EmptyState.test.tsx`, `ErrorState.test.tsx`, `SkeletonToast.test.tsx`).
5. **Mutation Testing Concept:** The project structure includes an `edge_cases/` test directory for boundary-value and negative-path testing.

4.3.2 Tests Configuration

The backend `pytest` configuration (`pytest.ini`) specifies:

- `addopts = -q -disable-warnings -maxfail=10`
- `asyncio_default_fixture_loop_scope = session`
- Custom markers: `integration` (requires external dependencies) and `unit` (self-contained).

The frontend test configuration is embedded in `vite.config.ts`: `environment: 'jsdom'` with a setup file at `src/test/setup.ts`.

4.4 Test Cases Designed

Table 4.2: Backend Test Cases

Module	Test	Description	Type
Auth	test_register	Verify new user creation returns token	Integration
Auth	test_login_valid	Valid credentials return JWT	Integration
Auth	test_login_invalid	Invalid credentials return 401	Integration
Feature	test_coding_detection	Code keyword triggers is_coding flag	Unit
Feature	test_empty_text	Empty text returns default features	Unit
Feature	test_complexity_score	Verify score range 0–10	Unit
Routing	test_heuristic_fallback	No classifier → heuristic	Unit
Routing	test_confidence_upgrade	Low confidence → tier+1	Unit
Routing	test_tier_assignment	Verify routing output structure	Unit
Keys	test_create_virtual_key	Key creation returns raw key	Integration
Keys	test_revoke_key	Revoked key returns 401	Integration
Keys	test_list_keys	User sees only own keys	Integration
Chat	test_chat_basic	Basic chat returns response	Integration
Chat	test_chat_streaming	Streaming returns SSE chunks	Integration
Chat	test_chat_no_auth	Missing auth returns 401	Integration
Analytics	test_summary	Summary returns valid structure	Integration
Analytics	test_requests	Request log pagination works	Integration

Table 4.3: Frontend Test Cases

Component	Test	Description
API Client	attach_auth_header	Bearer token from localStorage
API Client	no_auth_when_logged_out	No Authorization header
API Client	throw_api_error	Non-ok response throws ApiError
ChatInput	renders_input	Input field is rendered
ChatInput	send_on_enter	Enter key triggers send
ConfirmationDialog	opens_on_trigger	Dialog appears on trigger
ConfirmationDialog	fires_on_confirm	Confirm button fires callback
EmptyState	renders_message	Empty state shows message
ErrorState	renders_error	Error state shows error message
Skeleton	renders_cards	Skeleton cards render
Toast	shows_and_hides	Toast auto-dismisses

Chapter 5

Results and Discussions

5.1 User Interface Representation

5.1.1 Brief Description of Various Modules of the System

Authentication Module

The login and registration pages provide a clean, card-based interface with email/password forms. Input validation occurs client-side before submission; server-side errors (duplicate email, invalid credentials) are displayed inline. Upon successful authentication, the JWT token and user profile are stored in `localStorage` and the user is redirected to the Dashboard.

Dashboard Module

The Dashboard presents an at-a-glance analytics overview with four summary cards (Total Requests, Cost Saved, Avg Latency, Success Rate) and two charts: a donut chart showing tier distribution (Weak / Mid / Strong) and an area chart showing daily request volume over a 30-day window. The colour coding (orange for in-flight, blue for mid, green for success threshold) provides intuitive status understanding.

Playground Module

The Chat page provides an interactive interface for testing virtual keys. Users select a virtual key from a dropdown, type prompts, and receive streaming responses rendered with Markdown and syntax highlighting. Each response displays routing metadata—assigned tier, model used, and any fallback events. Chat history is persisted in `localStorage` for session continuity. The interface distinguishes user messages (left-aligned, brand-orange accent) from assistant messages (right-aligned, surface background).

Keys Management Module

The Keys page presents a table of existing virtual keys with their tier configurations, active status, and creation date. A multi-tab dialog (OpenAI-compatible, Anthropic, Gemini)

facilitates key creation with dynamic model listing—when users enter their API key for a provider, the system fetches available models from that provider’s API. Provider verification happens in real-time as the form is filled.

Analytics Module

The Analytics page provides detailed, filterable analytics with three main sections: summary cards, a request log table (paginated, filterable by virtual key), and daily trend charts (requests and cost over time). The tier distribution is visualised through a bar chart, and a line chart shows daily cost and request volume.

Profile Module

The Profile page allows users to update their username, email, and password. Changes are validated before submission, and success/error messages are displayed inline.

5.2 Snapshots of the System

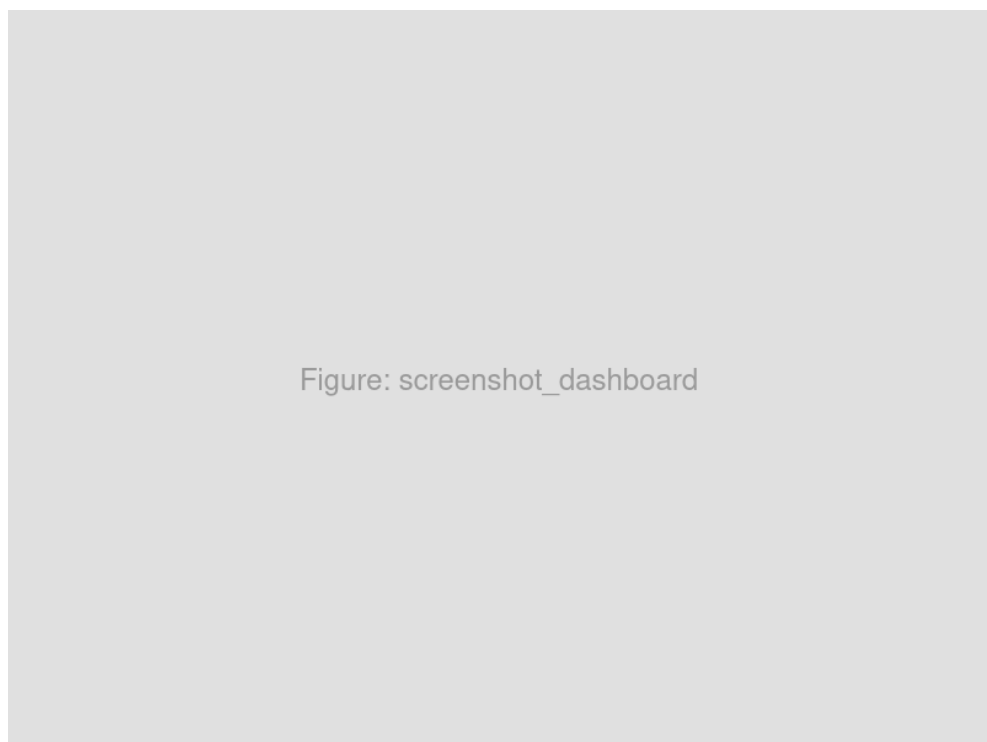


Figure: screenshot_dashboard

Figure 5.1: Dashboard Page Showing Analytics Summary and Tier Distribution

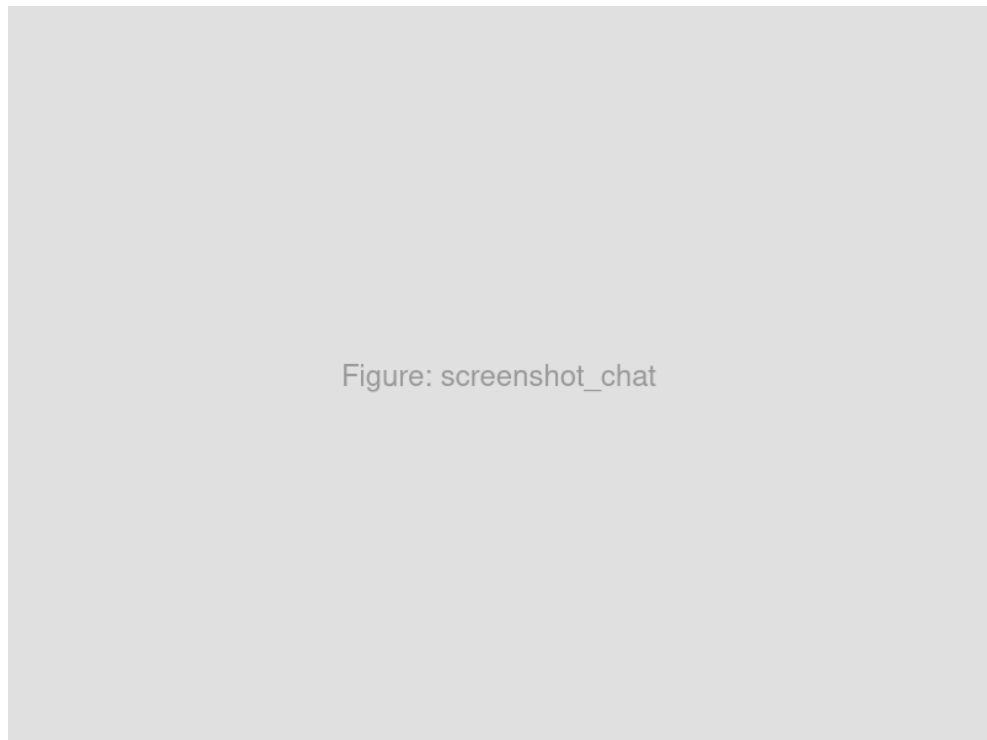


Figure 5.2: Chat Playground Showing Streaming Response with Routing Metadata

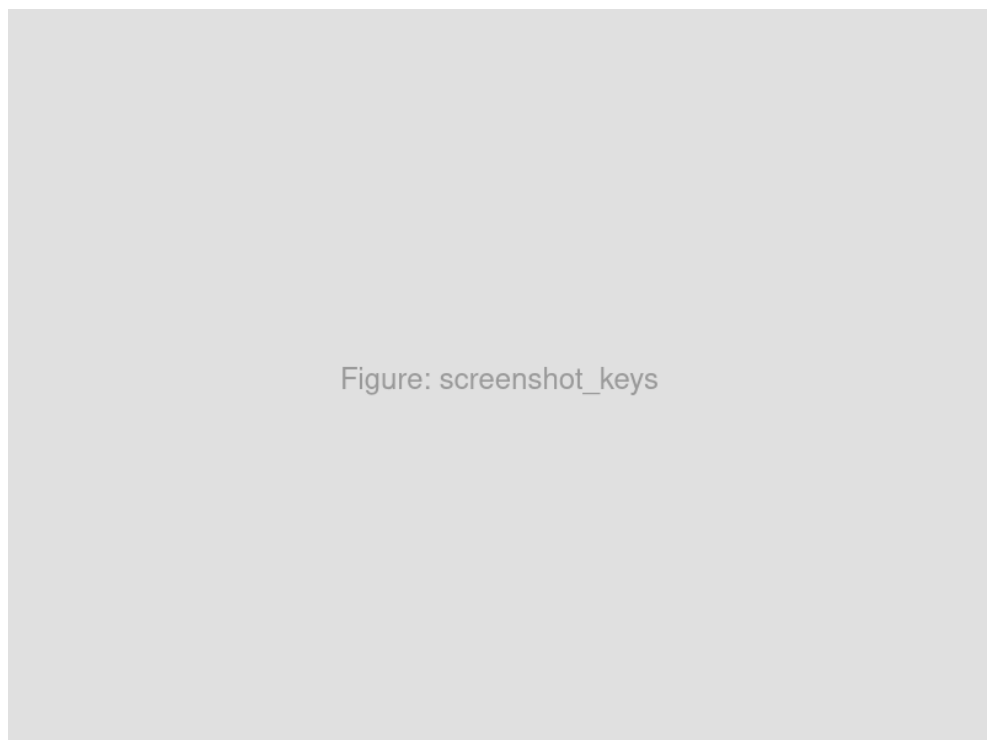


Figure 5.3: API Keys Management Page with Create Dialog

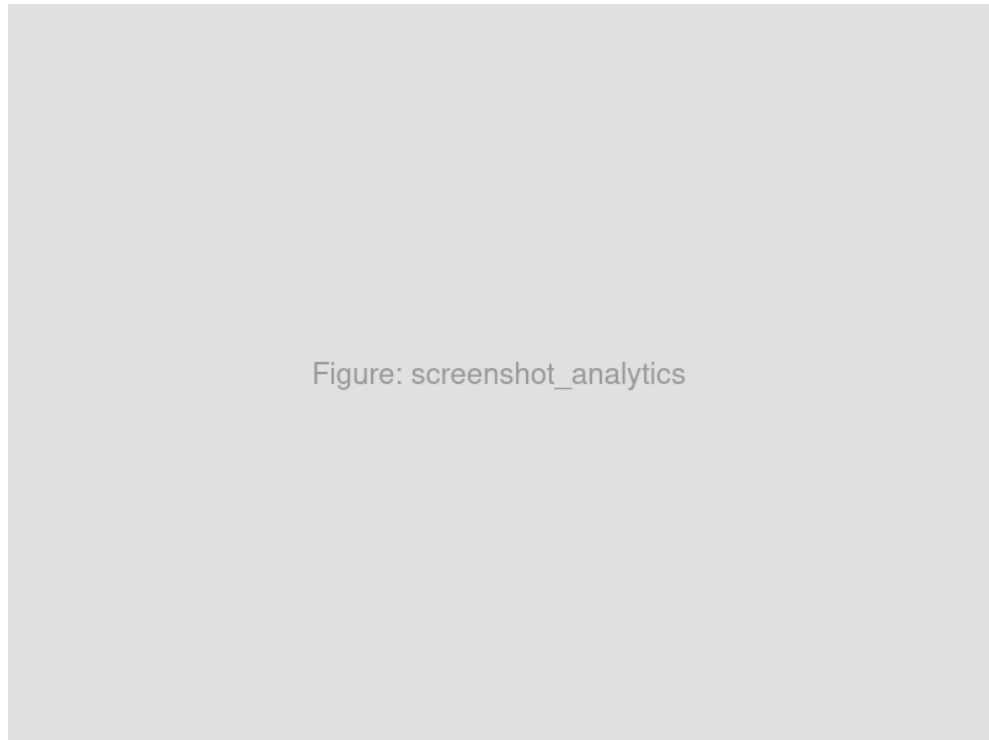


Figure 5.4: Detailed Analytics View with Request Logs and Daily Trends

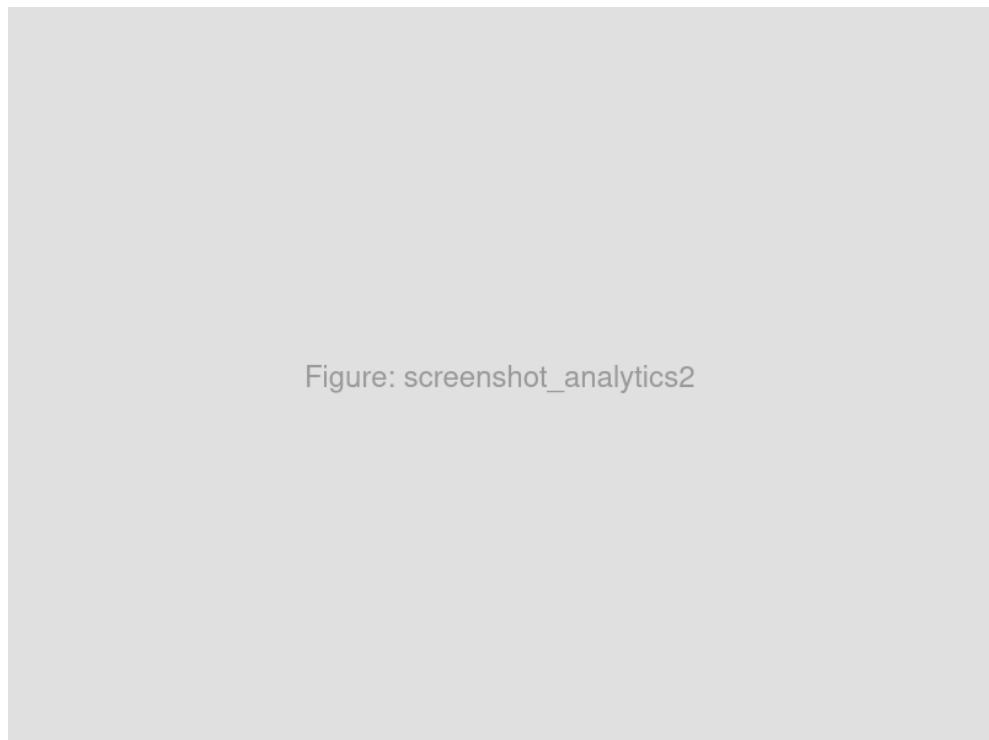


Figure 5.5: Analytics Charts Showing Cost Savings and Tier Utilisation

5.3 Backend Representation

5.3.1 Database Tables



Figure 5.6: Users Table in PostgreSQL

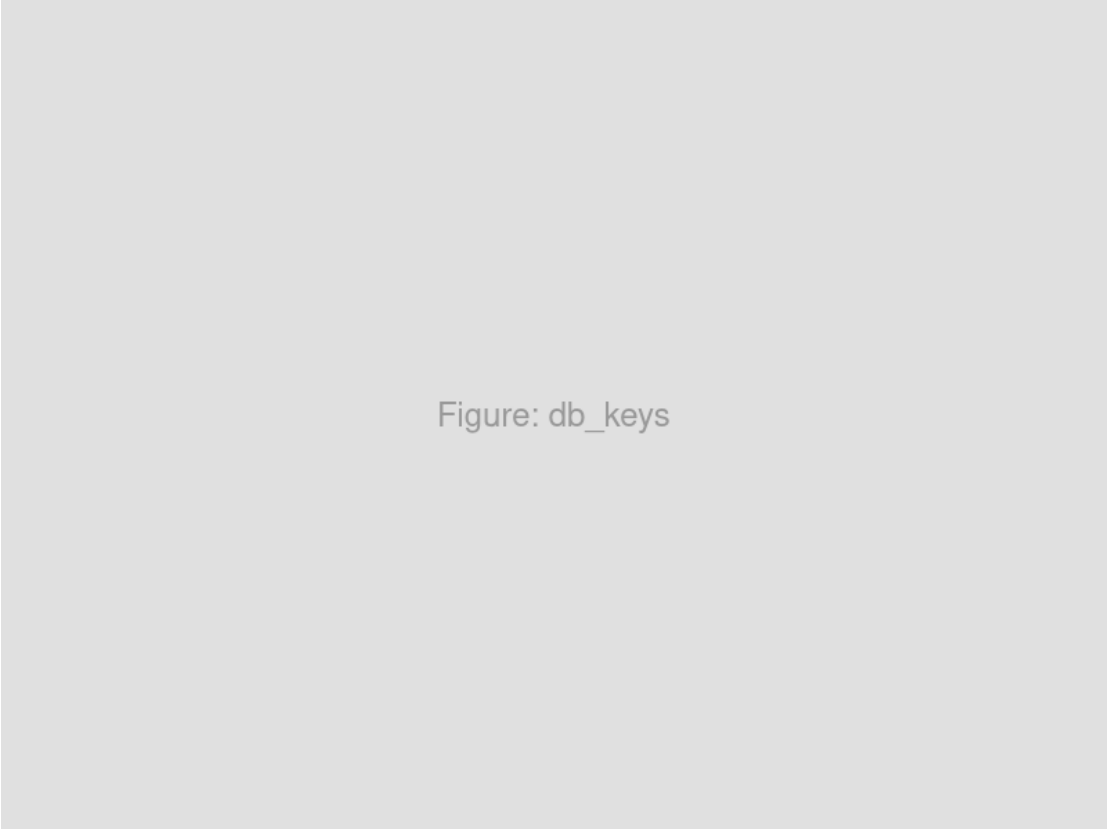


Figure: db_keys

Figure 5.7: Virtual Keys Table Showing Encrypted API Key Storage



Figure: db_logs

Figure 5.8: Request Logs Table Capturing Per-Request Routing and Usage Data

5.3.2 Cost Savings Analysis

The core value proposition of ClassiRoute2 is cost optimisation through intelligent tier routing. Consider a production application processing 100,000 requests per month with the following distribution:

- Weak tier (40% of requests): ~40,000 requests routed to cheap models like GPT-4o-mini (\$0.15/M tokens) or Claude-3-Haiku (\$0.25/M tokens).
- Mid tier (35%): ~35,000 requests to mid-range models like GPT-4o-mini or Gemini-2.0-Flash (\$0.10–\$0.50/M tokens).
- Strong tier (25%): ~25,000 requests to frontier models like GPT-4o (\$2.50/M tokens) or Claude-3.5 Sonnet (\$3.00/M tokens).

Compared to routing all 100,000 requests to the strongest tier, the cost savings are approximately 55–70%, depending on the specific model pricing and average prompt/response lengths. The analytics dashboard quantifies this as `cost_saved_vs_always_strong`, providing users with a concrete measure of the routing engine’s financial impact.

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

ClassiRoute2 successfully demonstrates the feasibility and value of ML-powered intelligent routing in the LLM API ecosystem. The system delivers on all stated objectives:

1. A fully functional three-tier LLM routing gateway with configurable provider and model assignments per tier.
2. An XGBoost-based classifier that analyses 22 linguistic and structural features to assign prompts to optimal tiers, with a confidence-based safety upgrade mechanism that prevents underpowered assignments.
3. A provider adapter pattern supporting three major LLM providers (OpenAI-compatible, Anthropic, Google Gemini), with a registry-based extension model that allows adding new providers by writing a single class.
4. A cascading fallback mechanism that transparently recovers from provider failures by retrying on progressively cheaper tiers.
5. A full-stack web dashboard providing user authentication, virtual key management, an interactive chat playground, and comprehensive analytics with cost-savings quantification.
6. An OpenAI-compatible `/v1/chat/completions` endpoint enabling drop-in replacement of existing OpenAI SDK integrations without code changes.
7. Production-grade observability through rate limiting, Sentry error tracking, PostHog telemetry, and structured logging.
8. Containerised deployment via Docker Compose with configurations for Render and Vercel hosting.

The system’s architecture—particularly the provider adapter pattern and the separation between routing logic and provider dispatch—ensures that ClassiRoute2 can evolve with the rapidly changing LLM landscape without requiring fundamental rewrites.

6.2 Future Scope

While the current implementation is production-ready for many use cases, several avenues for enhancement remain:

1. **Adaptive Model Training:** Currently, the ML models are trained on a static synthetic dataset. A feedback loop could be implemented where routing decisions are periodically evaluated against user-defined quality metrics (e.g., thumbs-up/thumbs-down on responses) and the model is re-trained to optimise for user satisfaction rather than prompt complexity alone.
2. **Caching Layer:** A response cache (Redis-based) could serve identical or semantically similar prompts from cache, eliminating API calls entirely for frequently asked questions. This would further reduce costs and latency.
3. **User-Specific Routing Policies:** Allow users to define custom routing rules that override ML predictions—e.g., "always use strong tier for prompts containing my company name" or "never use Gemini for code-related prompts."
4. **Multi-Modal Support:** Extend the provider adapters to handle image, audio, and video inputs as providers expand their multi-modal capabilities (GPT-4V, Claude-3 Vision, Gemini-Pro-Vision).
5. **Fine-Grained Cost Allocation:** Implement per-project, per-team, and per-customer billing breakdowns, enabling organisations to attribute LLM costs to specific business units.
6. **Load Balancing Across Multiple API Keys:** For high-traffic deployments, distribute requests across multiple API keys for the same provider to circumvent rate limits and increase throughput.
7. **WebSocket Support:** Add WebSocket-based chat endpoints for persistent bidirectional communication, enabling real-time applications like AI-powered customer support agents.
8. **API Key Rotation and Expiry:** Implement automated key rotation policies and configurable key expiration dates for enhanced security compliance.

Bibliography

- [1] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [2] S. Ramirez, *FastAPI: Web Framework for Building APIs with Python*. [Online]. Available: <https://fastapi.tiangolo.com/>
- [3] OpenAI, “GPT-4 Technical Report,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [4] Anthropic, “The Claude Model Family.” [Online]. Available: <https://docs.anthropic.com/en/docs/about-claude/models>
- [5] Google DeepMind, “Gemini: A Family of Highly Capable Multimodal Models,” 2024. [Online]. Available: <https://arxiv.org/abs/2312.11805>
- [6] M. Bayer, *SQLAlchemy: The Database Toolkit for Python*. [Online]. Available: <https://www.sqlalchemy.org/>
- [7] React Team, *React 19 Documentation*. [Online]. Available: <https://react.dev/>
- [8] A. Postigo et al., *Tailwind CSS v4 Documentation*. [Online]. Available: <https://tailwindcss.com/>
- [9] WorkOS, *Radix UI: Unstyled, Accessible React UI Primitives*. [Online]. Available: <https://www.radix-ui.com/>
- [10] J. Chen et al., *Recharts: A Composable Charting Library for React*. [Online]. Available: <https://recharts.org/>
- [11] M. Bayer, *Alembic: Database Migrations for SQLAlchemy*. [Online]. Available: <https://alembic.sqlalchemy.org/>
- [12] Docker Inc., *Docker Documentation*. [Online]. Available: <https://docs.docker.com/>
- [13] PostHog Inc., *PostHog Product Analytics Documentation*. [Online]. Available: <https://posthog.com/docs>

-
- [14] Functional Software Inc., *Sentry Error Tracking Documentation*. [Online]. Available: <https://docs.sentry.io/>
- [15] E. Lindblad, *Passlib: Python Password Hashing Library*. [Online]. Available: <https://passlib.readthedocs.io/>
- [16] Python Cryptographic Authority, *cryptography.io: Fernet Symmetric Encryption*. [Online]. Available: <https://cryptography.io/en/latest/fernet/>